ReconX Documentation (Detailed)

Generated: 2026-02-10

1. Project Summary

ReconX is a Linux-first Attack Surface Management (ASM) framework that orchestrates multiple recon, OSINT, and vulnerability scanning tools across four phases. A Python orchestrator runs phase-specific Bash modules, parses outputs, and stores structured results in SQLite for querying.

Primary goals:

- Broad coverage through tool redundancy.
- Repeatable phase-driven workflows with output organization.
- Local, queryable storage of results.

2. High-Level Architecture

Components:

- `reconx.py` orchestrates phases, threads targets, runs Bash modules, and parses outputs.
- Bash modules (`modules/*.sh`) run external tools and save outputs by phase.
- `parsers/parser.py` converts raw outputs into structured records.
- `db/database.py` stores results and scan progress in SQLite (WAL mode).
- `query.py` provides a CLI for reporting and exporting data.

Execution flow:

1. User chooses targets and phases.
2. `reconx.py` runs the phase module scripts.
3. Each module writes raw outputs into the phase directory.
4. Python parsers extract structured data.
5. SQLite stores results and progress.
6. `query.py` or SQL reads and reports.

3. Repository Layout

```
/Users/rejenthompson/Documents/technieum-/kali-linux-asm/
├── reconx.py
├── query.py
├── setup.sh
├── install.sh
├── config.yaml
├── requirements.txt
├── modules/
│   ├── 01_discovery.sh
│   ├── 02_intel.sh
```

```
│   ├── 03_content.sh
│   └── 04_vuln.sh
├── parsers/
│   └── parser.py
├── db/
│   └── database.py
├── examples/
│   └── quick_start.sh
├── output/
├── logs/
└── README.md
```

4. Installation and Setup

4.1 Quick Setup

- `setup.sh` creates directories, installs Python deps, and checks core tools.
- It assumes a `.env.example` template, but the file is missing in this repo (see Known Issues).

Command:

```
bash setup.sh
```

4.2 Full Install (Tools + System Dependencies)

`install.sh` installs system packages and a large toolset into `/opt` and `$HOME/go/bin`.

Command:

```
sudo bash install.sh
```

4.3 Python Dependencies

`requirements.txt` includes optional packages for querying and parsing. Install with:

```
pip3 install -r requirements.txt
```

5. Configuration

5.1 Actual Runtime Configuration (In Code)

The runtime behavior is mostly controlled by environment variables and hard-coded defaults in the Bash modules and `reconx.py`.

Key environment variables:

API keys:

- `SHODAN_API_KEY`
- `CENSYS_API_ID`
- `CENSYS_API_SECRET`
- `GITHUB_TOKEN`
- `SECURITYTRAILS_API_KEY`
- `PASTEBIN_API_KEY`

Orchestrator:

- `RECONX_CONTINUE_ON_FAIL`
- `RECONX_PHASE_TIMEOUT`
- `RECONX_PHASE1_TIMEOUT`
- `RECONX_PHASE2_TIMEOUT`
- `RECONX_PHASE3_TIMEOUT`
- `RECONX_PHASE4_TIMEOUT`

Phase 1 tuning:

- `RECONX_WHOIS_TIMEOUT`
- `RECONX_AMASS_INTEL_TIMEOUT`
- `RECONX_AMASS_ENUM_TIMEOUT`
- `RECONX_SUBLIST3R_TIMEOUT`
- `RECONX_SUBFINDER_TIMEOUT`
- `RECONX_SUBFINDER_THREADS`
- `RECONX_ASSETFINDER_TIMEOUT`
- `RECONX_SUBDOMINATOR_TIMEOUT`
- `RECONX_CRTSH_TIMEOUT`
- `RECONX_SECURITYTRAILS_TIMEOUT`
- `RECONX_DNSBRUTER_TIMEOUT`
- `RECONX_DNSPROBER_TIMEOUT`
- `RECONX_DNSX_TIMEOUT`
- `RECONX_DNSX_THREADS`
- `RECONX_HTTPX_TIMEOUT`
- `RECONX_HTTPX_THREADS`
- `RECONX_HTTPX_RUN_TIMEOUT`
- `RECONX_CERTSPOTTER_TIMEOUT`
- `RECONX_CT_MONITOR_TIMEOUT`
- `RECONX_ASNMAP_TIMEOUT`
- `RECONX_MAPCIDR_TIMEOUT`
- `RECONX_CLOUD_ENUM_TIMEOUT`
- `RECONX_S3SCANNER_TIMEOUT`
- `RECONX_GOBLOB_TIMEOUT`
- `RECONX_GCPBRUTE_TIMEOUT`
- `RECONX_CLOUD_THREADS`

- `RECONX_CLOUD_KEYWORDS_LIMIT`

Phase 2 tuning:

- `RECONX_RUSTSCAN_BATCH`
- `RECONX_RUSTSCAN_TIMEOUT`
- `RECONX_SUBJACK_THREADS`
- `RECONX_NMAP_MAX_HOSTS`
- `RECONX_NMAP_HOST_TIMEOUT`
- `RECONX_NMAP_MAX_FILE_MB`
- `RECONX_MIN_DISK_MB`

Phase 3 tuning:

- `RECONX_GAU_THREADS`
- `RECONX_GOSPIDER_THREADS`
- `RECONX_GOSPIDER_CONCURRENCY`
- `RECONX_KATANA_CONCURRENCY`
- `RECONX_FFUF_THREADS`
- `RECONX_FFUF_TIMEOUT`
- `RECONX_FEROX_THREADS`
- `RECONX_DIRSEARCH_THREADS`

Phase 4 tuning:

- `RECONX_THREADS`
- `RECONX_SQLMAP_THREADS`
- `RECONX_VULN_TIMEOUT`
- `RECONX_NUCLEI_RATE_HIGH`
- `RECONX_NUCLEI_RATE_MED`
- `RECONX_NUCLEI_RATE_LOW`
- `RECONX_NUCLEI_RATE_CVE`
- `RECONX_NUCLEI_RATE_MISC`
- `RECONX_SKIPFISH_TIME`
- `RECONX_SKIPFISH_RPS`
- `RECONX_SKIPFISH_MAX_HOSTS`
- `RECONX_SKIPFISH_CONN_GLOBAL`
- `RECONX_SKIPFISH_CONN_PERIP`
- `RECONX_SKIPFISH_REQ_TIMEOUT`
- `RECONX_SKIPFISH_DEPTH`
- `RECONX_SKIPFISH_CHILDREN`
- `RECONX_SKIPFISH_MAX_REQUESTS`
- `RECONX_SKIPFISH_MAX_DESC`
- `RECONX_SKIPFISH_PARTIAL`
- `RECONX_SKIPFISH_RESP_SIZE`

5.2 config.yaml Status

`config.yaml` is present but not loaded anywhere in the Python or Bash code. Settings in this file currently do not affect runtime behavior.

6. Usage

6.1 ReconX Orchestrator

```
python3 reconx.py -t example.com
python3 reconx.py -t example.com,example.org -T 5
python3 reconx.py -f targets.txt
python3 reconx.py -t example.com -p 1,2
```

CLI arguments:

- `-t, --target` comma-separated domains
- `-f, --file` file with one domain per line
- `-o, --output` output directory
- `-d, --database` database path
- `-p, --phases` phases to run
- `-T, --threads` number of concurrent targets
- `--resume` declared but currently unused in code

6.2 Query Tool

```
python3 query.py --list
python3 query.py -t example.com --summary
python3 query.py -t example.com --subdomains --alive-only
python3 query.py -t example.com --vulns --severity critical
python3 query.py -t example.com --export vulnerabilities -o vulns.csv
```

7. Output Structure

Example layout for one target:

```
output/
└── example_com/
    ├── phase1_discovery/
    ├── phase2_intel/
    ├── phase3_content/
    └── phase4_vulnscan/
```

Each phase produces subfolders and raw tool outputs. Parsers read selected files and insert structured records into `reconx.db`.

8. Database Schema (SQLite)

Tables created in `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/db/database.py`:

- `scan_progress` stores phase completion status per target.
- `tool_runs` stores tool execution status but is not currently used by code.
- `acquisitions` intended for company/domain relationships but not used.
- `subdomains` stores discovered hosts with optional IP and alive status.
- `ports` stores open ports and service metadata.
- `urls` stores discovered URLs and their source tool.
- `leaks` stores secret findings.
- `vulnerabilities` stores findings from scanners.
- `infrastructure` intended for OSINT/infrastructure data but not used.

9. Phase-by-Phase Details

This section lists actual tools executed by the Bash modules and whether results are parsed into the database.

9.1 Phase 1: Discovery and Enumeration

Inputs:

- Target domain

Tools executed:

- `whois`
- `amass` (intel or passive enum)
- `getSubsidiaries` (optional, from `/opt`)
- `sublist3r`
- `assetfinder`
- `subfinder`
- `subdominator`
- `crt.sh` (via curl + jq)
- `certspotter` API (via curl + jq)
- `ct-monitor` (optional)
- `securitytrails` API (if key set)
- `dnsbruter` (optional)
- `dnsprober` (optional)
- `asnmap` + `mapcidr` (ASN expansion)
- `dnsx` (resolution)
- `httpx` (live host validation)
- `cloud_enum`, `s3scanner`, `goblob`, `gcpbucketbrute` (cloud exposure checks)

Primary outputs:

- `phase1_discovery/all_subdomains.txt`

- `phase1_discovery/resolved_subdomains.txt`
- `phase1_discovery/alive_hosts.txt`
- `phase1_discovery/httpx_alive.json`
- `phase1_discovery/dnsx_resolved.json`

Parsed into DB:

- Subdomains (from `all_subdomains.txt`, `passive_subdomains.txt`, `active_subdomains.txt`)
- Alive hosts (from `httpx_alive.json` or `alive_hosts.txt` fallback)
- DNS resolution (from `dnsx_resolved.json`)

Not parsed:

- Whois output
- Subsidiary results
- CT sources
- ASN and cloud exposure outputs

9.2 Phase 2: Intelligence and Infrastructure

Inputs:

- `phase1_discovery/alive_hosts.txt`

Tools executed:

- `subprober` (validation)
- `rustscan`
- `nmap`
- `shodan` CLI
- `shodanx`
- `googledorker` (dorker)
- `censys` CLI
- `whois` ASN lookup
- `subjack`
- `subover`
- `gitleaks`
- `githunt`
- `trufflehog`
- `git-secrets`

Primary outputs:

- `phase2_intel/ports/nmap_all.xml`
- `phase2_intel/ports/rustscan_ports.txt`
- `phase2_intel/takeover/subjack_results.txt`
- `phase2_intel/leaks/gitleaks_*.json`

Parsed into DB:

- Open ports (from Nmap XML and RustScan)
- Takeover findings (from Subjack)
- Leaks (from Gitleaks JSON)

Not parsed:

- SubProber validation output
- Shodan / Censys / ShodanX data
- Google dorking output
- SubOver results
- GitHunt / TruffleHog / git-secrets outputs
- ASN lookup results

9.3 Phase 3: Deep Web and Content Discovery

Inputs:

- `phase1_discovery/alive_hosts.txt`
- `phase1_discovery/all_subdomains.txt`

Tools executed:

- `gau`, `waybackurls`
- `spideyx`, `gospider`, `hakrawler`, `katana`
- `ffuf`, `feroxbuster`, `dirsearch`
- `linkfinder`, `secretfinder`, `jsscanner`
- `PasteHunter`, Pastebin API, `pbin`
- `newman`, `kiterunner`, `arjun`

Primary outputs:

- `phase3_content/urls/all_urls.txt`
- `phase3_content/urls/javascript_files.txt`
- `phase3_content/bruteforce/ffuf_all.json`
- `phase3_content/javascript/linkfinder_endpoints.txt`
- `phase3_content/javascript/secretfinder_secrets.txt`

Parsed into DB:

- URLs from `gau`, `waybackurls`, `spideyx`, `gospider`, `hakrawler`, `katana`
- FFUF results (intended, but currently broken; see Known Issues)
- JS secrets and endpoints from SecretFinder and LinkFinder

Not parsed:

- Feroxbuster and Dirsearch results
- SpideyX JS scraper output
- JSScanner output
- Pastebin monitoring outputs
- Newman, Kiterunner, Arjun outputs

9.4 Phase 4: Vulnerability Scanning

Inputs:

- `phase1_discovery/alive_hosts.txt`
- `phase3_content/urls/all_urls.txt`

Tools executed:

- `nuclei` (multiple severity scans)
- `dalfox`
- `xsstrike` (optional)
- `sqlmap`
- `corsy`
- `nikto`, `wpscan`, `wapiti`, `skipfish`, `cmsmap`, `retire`
- `testssl.sh`, `sslyze`

Primary outputs:

- `phase4_vulnscan/nuclei/nuclei_all.json`
- `phase4_vulnscan/xss/dalfox_results.txt`
- `phase4_vulnscan/sqli/sqlmap_results.txt`
- `phase4_vulnscan/cors/corsy_results.txt`

Parsed into DB:

- Nuclei findings
- Dalfox findings
- SQLMap findings
- Corsy findings

Not parsed:

- XSStrike output
- Nikto, WPScan, Wapiti, Skipfish, CMSmap outputs
- Retire.js output
- testssl.sh / SSLyze outputs

10. Verified Implementations (What Works)

These features are directly implemented in code and connected end-to-end.

- Phase orchestration and per-target threading in `reconx.py`.
- Output directory creation and per-phase structure.
- Phase execution via `modules/*.sh`.
- SQLite database creation with WAL mode and indexes.
- Subdomains from phase 1 lists are parsed and inserted.
- DNSx and HTTPx results are parsed and inserted.
- RustScan and Nmap results are parsed and inserted.
- Gitleaks findings are parsed and inserted.

- URL discovery from gau, waybackurls, spideyx, gospider, hakrawler, and katana is parsed and inserted.
- Nuclei, Dalfox, SQLMap, and Corsy findings are parsed and inserted.
- Query tool summaries and CSV export.

11. Known Issues and Parts Not Working Properly

These are code-level gaps or inconsistencies that affect correctness or expected behavior.

- `config.yaml` is never loaded, so all settings in it are ignored. Reference: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/config.yaml` and `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/reconx.py`.
- The `--resume` CLI flag is parsed but unused. Reference: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/reconx.py`.
- `setup.sh` expects `.env.example`, but this file does not exist in the repo. Reference: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/setup.sh`.
- `insert_subdomains_bulk` uses `INSERT OR IGNORE` and does not update existing rows. This prevents HTTPx/DNSx results from updating `is_alive`, `ip`, and `status_code` for subdomains already inserted earlier. Reference: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/db/database.py`.
- FFUF parsing is broken for the current output format. The module concatenates multiple JSON objects into `ffuf_all.json`, but `read_json` only handles JSON arrays or JSONL. This likely produces zero parsed FFUF results. References: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/modules/03_content.sh` and `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/parsers/parser.py`.
- `tool_runs`, `acquisitions`, and `infrastructure` tables are never populated by the code. Reference: `/Users/rejenthompson/Documents/technieum-/kali-linux-asm/db/database.py`.
- Many tool outputs are generated but never parsed into the database, reducing the value of `query.py` reports. See Phase details above.

12. Extra Features That Can Be Implemented Next

These are high-impact improvements that align with current structure.

- Load `config.yaml` and map its values to environment variables or direct function parameters.
- Implement `.env` loading using `python-dotenv` so API keys are automatically read.
- Fix FFUF parsing by reading each `ffuf_*.json` file individually or using a JSONL exporter.
- Update `insert_subdomains_bulk` to upsert and preserve `is_alive`, `ip`, and `status_code`.
- Parse SubProber results to update alive status.
- Parse Feroxbuster and Dirsearch results into `urls`.
- Parse Shodan and Censys outputs into `infrastructure`.
- Parse cloud exposure results into a new table or `infrastructure`.
- Parse outputs from additional scanners such as Nikto, WPScan, Wapiti, Skipfish, CMSmap, Retire.js, and SSL tools.

- Implement tool execution tracking using `tool_runs` with per-tool statuses and durations.
- Add report generation (Markdown or HTML) summarizing findings by severity and asset class.
- Add `--force` or `--rerun` flags to override completed phases cleanly.
- Add unit tests for parsers and DB insertions to prevent silent data loss.

13. Security and Legal Notice

ReconX is a security testing framework. Use it only on assets you own or have explicit authorization to test. Unauthorized scanning is illegal in many jurisdictions.

14. Troubleshooting

- Missing tools: run `sudo bash install.sh` and ensure `$GOPATH/bin` is in `PATH`.
- SQLite locked: `sqlite3 reconx.db "PRAGMA wal_checkpoint(TRUNCATE);"`
- Rate limits: reduce tool concurrency via environment variables.
- No results: verify DNS resolution, tool availability, and API keys.

15. Quick Reference Commands

```
# Run all phases
python3 reconx.py -t example.com

# Run only discovery and intel
python3 reconx.py -t example.com -p 1,2

# Query summary
python3 query.py -t example.com --summary

# Export vulnerabilities
python3 query.py -t example.com --export vulnerabilities -o vulns.csv
```